

La programmation par IA remplacera-t-elle tous les programmeurs ? Voyons la situation réelle du secteur

Zhihu · 2026-09-19

Depuis la fin 2025 et le début 2026, les performances de la programmation par IA se sont tellement améliorées qu'une grande partie, voire l'essentiel, du travail de codage des programmeurs est désormais pris en charge par l'IA. Ce phénomène a suscité panique et désespoir dans la profession. Dans le même temps, les entreprises d'IA exploitent ces progrès pour exagérer sans limite les capacités de l'IA, proclamant que tous les cols blancs seront licenciés d'ici un ou deux ans. Les médias personnels s'engouffrent dans la brèche, diffusant la panique et fabriquant de l'anxiété.

Il nous faut examiner sérieusement la question. La programmation par IA peut-elle remplacer tous les programmeurs ? Et si non, quel est le taux de remplacement réel ?

En réalité, dans une réponse écrite il y a quelques mois (Le premier coup d'épée de l'IA frappe les programmeurs : vont-ils être massivement au chômage ?), j'avais livré mon jugement : l'IA ne peut pas remplacer l'ensemble du travail humain, le taux de remplacement de bout en bout ne dépassant pas 30 %.

Lorsque les systèmes de machines intelligentes ayant l'IA pour cœur entreront massivement dans la production, le travail de codage humain disparaîtra pour l'essentiel. Bien sûr, le développement de code critique, à l'architecture complexe ou aux exigences de performance très élevées, pourra encore nécessiter une forte intervention humaine ; mais l'immense masse du développement de code ordinaire sera entièrement prise en charge par l'IA. Combien de personnes cela remplacera-t-il concrètement ? D'après mes propres résultats d'usage, et d'après les retours d'autres personnes sur Zhihu, le gain de productivité va d'environ 20 % à 50 %. Si la programmation par IA se déploie normalement (on en est encore à la phase d'expérimentation massive dans le secteur) et si l'on estime à 60 % la part des développeurs purs, on aboutit à une réduction d'environ 12 % à 30 % des postes de programmeurs.

Le déploiement à grande échelle de la programmation par IA dure depuis au moins six mois, et notamment en Chine et aux États-Unis, où les grandes entreprises exigent quasiment toutes de leurs salariés qu'ils développent avec des agents d'IA. Quel est donc le résultat effectif ?

Selon un rapport de McKinsey^[1], chez les programmeurs qui « savent utiliser l'IA », le gain de productivité serait d'environ 16 %~30 %, ce qui correspond dans les grandes lignes à mon jugement antérieur.

 Figure 1

Une étude récente^[2] montre elle aussi que, si la programmation par IA accélère fortement la génération de code, une fois rapporté à la publication finale des produits, le gain d'efficacité n'est que d'environ 30 %. Cela concorde là encore avec l'étude de McKinsey ci-dessus et avec ma supposition antérieure.

Les outils d'IA générative appliqués au développement logiciel progressent à une vitesse fulgurante : en quelques années à peine, ils sont passés de la simple autocomplétion de code à de véritables agents de codage autonomes (coding agents). Ces progrès se traduisent-ils par des gains de productivité au niveau des tâches dans le monde réel ? Et même si c'est le cas, l'amélioration de l'efficacité au niveau des tâches signifie-t-elle un accroissement de la production finale ? Ces questions sont essentielles pour comprendre l'impact économique de l'IA. À partir d'une étude d'événements appariés portant sur des développeurs, nous constatons que, à mesure que les outils de codage par IA s'améliorent, leurs effets de productivité au niveau des tâches deviennent plus marqués : l'autocomplétion a augmenté le nombre de commits hebdomadaires d'environ 30 %, les agents synchrones (sync agents) ont porté cet effet cumulé à environ 180 %, et les agents asynchrones (async agents) à environ 240 %. Pourtant, à mesure que l'on remonte vers la production, ces gains s'affaiblissent brutalement : alors que les agents synchrones augmentent le nombre de lignes de code de 958 % et les pull requests de 86 %, le nombre de versions effectivement publiées n'a progressé que de 20 %. L'activité de codage globale sur GitHub suit la même loi.

 Figure 2

Source : <https://mitsloan.mit.edu/ideas-made-to-matter/ai-boosts-worker-productivity-does-translate-to-final-outputs>

Non seulement diverses études confirment les limites de la programmation par IA, mais les développeurs et les managers qui utilisent réellement des agents d'IA le ressentent plus directement encore. Le regard des programmeurs sur la programmation par IA est revenu du choc initial à la lucidité : l'usage réel en production leur a fait peu à peu prendre conscience de ses limites. Ainsi, un billet de blog^[3] de Cal Newport (professeur d'informatique à l'université de Georgetown) raconte l'histoire d'un développeur qui — un ingénieur chevronné de la Silicon Valley, dont l'attitude s'est transformée au cours de ces six derniers mois — au début de l'année, après avoir programmé avec Claude Code, voyait sa productivité multipliée par plusieurs et s'écriait « really, really good », puis qui, après deux incidents de production causés par du code d'IA et avoir failli perdre son emploi, s'est désillusionné et s'est remis à écrire à la main le code critique, ne réservant l'IA qu'aux tests et aux scripts. Ce développeur estime que la programmation à l'ancienne est plus lente, mais fiable et de qualité garantie. L'auteur du billet indique que ce cas n'est pas isolé et que nombre d'ingénieurs du secteur expriment des vues semblables.

Où la programmation par IA a-t-elle mal ? Un programmeur en témoigne dans un billet de blog^[4], où il raconte ses impressions après six mois passés à examiner du code d'IA. Selon lui, un agent d'IA ressemble à un très bon nouvel employé : le code qu'il écrit paraît excellent, mais, faute de connaissances du domaine, il tient toujours insuffisamment compte des contraintes, des limites et des cas exceptionnels, et il est enclin à introduire des problèmes de sécurité, et une étude intitulée « Constraint Decay : la fragilité des agents LLM dans la génération de code backend »^[5] le documente.

Notre recherche met au jour un phénomène de « décroissance des contraintes » : à mesure que les exigences structurelles s'accumulent, les performances des agents chutent nettement. Des tâches de référence aux tâches entièrement spécifiées, les configurations les plus performantes voient leur taux de réussite des assertions baisser en moyenne de 30 points, et certaines configurations plus faibles tombent même près de zéro. L'analyse de sensibilité aux frameworks révèle des écarts marqués : les agents se comportent bien dans des frameworks minimalistes et explicites (comme Flask), mais leurs performances moyennes s'effondrent dans des environnements fortement conventionnels (comme FastAPI, Django). Enfin, l'analyse des erreurs montre que les défauts de la couche de données (construction incorrecte de requêtes, exceptions d'exécution de l'ORM, par exemple) en sont la source principale. Cette étude souligne que satisfaire simultanément les exigences fonctionnelles et structurelles reste un défi crucial et non résolu pour les agents de programmation.

Un directeur technique d'une entreprise indienne le relève lui aussi dans un article récent^[6]. Selon lui, le code écrit par l'IA est facilement sujet à des problèmes lors de l'exploitation réelle en production, parce que « dans les grands systèmes, le code doit respecter les schémas d'architecture, les chaînes de dépendances, les règles métier, les exigences de conformité, les contrôles de sécurité et les processus de publication. Si le code généré par l'IA ignore ces contraintes, le centre de gravité du travail se déplace du développement vers les réparations et les corrections ultérieures, ce qui allonge les cycles de relecture du code, laisse la couverture de tests insuffisante, fait découvrir des failles de sécurité, provoque des échecs d'intégration et des défauts en environnement de production, etc. » Quiconque a une expérience réelle de la programmation par IA comprendra que ces problèmes sont effectivement très répandus.

Le code d'IA s'écrit vite, mais en raison des problèmes exposés plus haut, les développeurs doivent examiner attentivement le code généré avant de pouvoir le fusionner dans le dépôt. D'où deux questions. Premièrement, la vitesse de relecture du code est nettement inférieure à la vitesse de génération, ce qui réduit fortement le gain de productivité de l'IA. Deuxièmement, les nouveaux programmeurs manquent d'occasions d'écrire du code de leurs mains et ne peuvent progresser ; une fois la vieille génération de programmeurs licenciée ou à la retraite, la relecture du code finira par ralentir davantage, au point de ne plus détecter les problèmes.

Plus grave encore, certaines entreprises placent de grands espoirs dans la programmation par IA et compriment fortement les délais de développement, au point que les programmeurs n'ont plus le temps de relire sérieusement le code. Pour un développeur, l'entreprise n'est pas la sienne : si le supérieur exige d'aller vite, il relira sommairement, voire fusionnera sans relire. Si des problèmes apparaissent en production, il partira, voilà tout. Fatigue de relecture ou relecture insuffisante : le résultat final est que le code mis en production recèle quantité de BUG et de code smells, et que ceux qui maintiennent ces logiciels ne connaissent en réalité pas le code. Toute cette dette technique produit ce qu'on appelle le « fossé de maintenabilité » (The Maintainability Gap)^[7].

La programmation assistée par IA ne cesse de progresser rapidement, et les développeurs les plus motivés produisent des volumes de code records. Pourtant, l'analyse de quatre années de données de modification du code montre que les indicateurs de maintenabilité se dégradent : les appels de fonctions entre fichiers (qui traduisent la réutilisation du code) ont reculé de 35 % ; par rapport au niveau de 2022, les migrations de lignes de code lors des refactorisations ont diminué de 70 %, et le travail de maintenance sur le code hérité de longue date a reculé de 74 %. Dans le même temps, nous observons une progression de phénomènes inquiétants : les comportements de « copier/coller » dans un même commit ont augmenté de 41 %, la duplication de blocs de code de 81 %, les structures de code qui masquent les erreurs de 47 %, et le taux de churn du code sur deux semaines de 15 %. La production de code augmente certes fortement, mais la dette technique qui s'accumule en même temps n'est pas négligeable ; cette dette pèse principalement sur les développeurs qui n'ont pas encore conscience que certains schémas de codage compromettent la maintenabilité à long terme du dépôt.

Concernant la dette technique, l'étude « Debt Behind the AI Boom »^[8], s'appuyant sur une vaste étude, établit que « les code smells sont le type de problème le plus fréquent, avec une part atteignant 89,3 % », que « parmi les commits produits par les divers assistants de programmation IA, plus de 15 % introduisent au moins un problème. Plus important encore, parmi les problèmes introduits par l'IA, 22,7 % subsistent dans la dernière version du dépôt », et que « ces constats indiquent que le code généré par l'IA peut imposer aux projets logiciels réels des coûts de maintenance durables, tout en soulignant la nécessité de renforcer l'assurance qualité dans le développement assisté par l'IA ».

 Figure 3

Tiré de : « Debt Behind the AI Boom »

Quant à l'étude « Faster Code, Deeper Debt? »^[9], elle montre que « les LLM ont tendance à aggraver les formes traditionnelles de dette technique (en particulier la dette de code, de conception et de documentation), tout en introduisant de nouvelles formes de dette propres aux LLM. Il faut noter en particulier une "dette d'intégration rapide" : la qualité sacrifiée à la vitesse fait que du code généré à la hâte déclenche des réactions en chaîne, engendrant une dette de gouvernance et un surcroît de coûts de maintenance à long terme. D'autres catégories de dette émergentes incluent la dette de

prompt, la dette éthique, la dette de données et la dette de traçabilité, qui reflètent les défis propres aux applications des LLM. »

La programmation par IA n'entraîne pas seulement l'accumulation d'une dette technique ; elle introduit aussi d'autres formes de dette. L'informaticienne Margaret-Anne Storey, dans son article « From Technical Debt to Cognitive and Intent Debt »^[10], selon qui, au-delà de la dette technique, « à mesure que la vitesse de génération de code par l'intelligence artificielle dépasse la vitesse à laquelle les équipes comprennent ce code, deux formes de « dette » souvent sous-estimées s'accumulent », en distingue deux : premièrement la « dette cognitive » (cognitive debt), c'est-à-dire la perte du consensus au sein de l'équipe ; deuxièmement la « dette d'intention » (intent debt), c'est-à-dire l'absence de justification logique explicitée, nécessaire pour que les développeurs et les agents d'intelligence artificielle traitent le code en sécurité. En langage courant : l'usage généralisé de la programmation par IA risque de faire que plus personne dans l'équipe ne sache ce qu'il fait. À ce jour, l'attention du secteur reste braquée sur la dette technique, et ces problèmes plus profonds ne se sont pas encore pleinement révélés.

Arrivé ici, j'imagine que le lecteur a déjà sa réponse. J'ai répété un point de vue dans bien des réponses (par exemple : la programmation par IA remplacera-t-elle vraiment les programmeurs à l'avenir ? et : comment considérer la prédiction du responsable de l'IA chez Microsoft selon laquelle « dans les 18 prochains mois, la plupart des emplois de cols blancs seront complètement remplacés par l'IA » ?) : l'IA transformera profondément la manière de travailler des cols blancs, mais elle ne pourra pas éliminer tous les programmeurs, et encore moins mettre au chômage tous les cols blancs. Les faits ont confirmé que mon point de vue était juste.

À mesure que se dissipe le mythe du remplacement de tous les cols blancs par l'IA, le marché des capitaux commence à montrer des signes d'éclatement de bulle. J'avais déduit, sur l'hypothèse d'un gain de productivité de 30 % dans le développement logiciel, que la bulle de l'IA existait bel et bien et que son ampleur dépassait celle de la bulle internet (voir : « Quelle est l'ampleur de la bulle de l'IA ? Sa crise dépassera-t-elle de loin la crise financière de 2008 ? »). Les théories de fin de l'humanité par l'IA récemment lâchées par les PDG des grandes entreprises de modèles d'IA ne sont que la dernière frénésie avant la chute.

Notes

^[1] La révolution de l'IA dans le développement logiciel

中文原文：The AI revolution in software development

[9] Code plus rapide, dette plus profonde ? Revue de littérature multivocale sur la dette technique et ses premiers signes dans le développement logiciel assisté par LLM

中文原文：Faster Code, Deeper Debt? A Multivocal Literature Review on Technical Debt and Its Early Signs in LLM-Assisted Software Development

<https://arxiv.org/abs/2606.14796>

[10] De la dette technique à la dette cognitive et d'intention : repenser la santé du logiciel à l'ère de l'IA

中文原文：From Technical Debt to Cognitive and Intent Debt: Rethinking Software Health in the Age of AI

<https://arxiv.org/abs/2603.22106>

Cet article a été publié à l'origine sur Zhihu (en chinois) : <https://zhuanlan.zhihu.com/p/2084561490930045240> Auteur : 最最遥远的路 · Profil Zhihu : <https://www.zhihu.com/people/zui-zui-yao-yuan-de-lu-32/posts>

本文原发表于知乎专栏/回答（中文）： <https://zhuanlan.zhihu.com/p/2084561490930045240>；作者：最最遥远的路： <https://www.zhihu.com/people/zui-zui-yao-yuan-de-lu-32/posts>